



U1 A1:

"PROGRAMA SIMPLE EN PYTHON"

TECNOLOGÍAS DE LA INFORMACIÓN E

INNOVACIÓN DIGITAL

ÁREA INFRAESTRUCTURA DE REDES DIGITALES

PRESENTA

GABRIEL GALARZA ALEMÁN

Índice

Tema	Página
"Programa simple en python"	i
Índice de Figuras.....	ii
Capítulo 1. Introducción.....	1
Capítulo 2: Nombre de Tema.....	2
2.1. Descripción del programa.....	2
2.2. Explicación del código.	2
Capítulo 3: Conclusión.....	6

Índice de Figuras

Figura	Página
Figura 1. Entrada de datos.	2
Figura 2. Lista de puertos activos.	3
Figura 3. Horas y días activos.....	3
Figura 4. Diccionario.	4
Figura 5. Estado actual del dispositivo.	4
Figura 6. Salida a archivo de texto.....	4
Figura 7. Reporte finalizado.....	5

Capítulo 1. Introducción.

En este documento, se explicará el proceso usado para realizar un simple programa en Python, el cual hará uso de diversas funciones del lenguaje, como declaración de variables, conversión de tipos de datos, listas, diccionarios, estructuras condicionales, y creación de archivos.

Capítulo 2: Nombre de Tema.

2.1. Descripción del programa

El programa elaborado crea un reporte de dispositivos de red en base a los datos ingresados por el usuario: pide información acerca de los dispositivos, y guarda los resultados a un archivo de texto.

2.2. Explicación del código.

El programa inicia con una simple declaración de variables, las cuales también reciben datos a través de una entrada de texto. En este caso, pide el nombre del dispositivo, y la dirección IP de este.

```
device_name = input("Insert device name: ")  
device_ip = input("Insert IP address: ")
```

Figura 1. Entrada de datos.

Después inicializa una lista, esta lista la usará para guardar un listado de los puertos activos en el dispositivo. Recibe entrada de los puertos (especificando que sean separados por ","), y un ciclo for los inserta uno por uno, convirtiendo la entrada a int en el proceso.

```
open_ports = []
open_ports_input = input("Insert active ports (comma-separated): ")
for ports in open_ports_input.split(","):
    open_ports.append(int(ports.strip()))
```

Figura 2. Lista de puertos activos.

A continuación, se pregunta si el dispositivo está activo. La razón de crear un if para esto es que, si el dispositivo no se encontrara activo, no sería posible que tuviera más de 0 horas de tiempo activo. Por tanto, solo se llega a preguntar esto si es que el dispositivo se encuentre activo.

Después de haber recibido el valor de las horas activas, se procede a obtener el valor de días activos, con una simple operación.

```
active_input = input("Is the device active? (Y/N): ").strip().lower()
if active_input in ("y", "yes", "1"):
    is_active = True
    uptime_hours = float(input("Insert device uptime (in hours): "))
elif active_input in ("n", "no", "0"):
    is_active = False
    uptime_hours = 0

uptime_days = round(uptime_hours / 24, 2)
```

Figura 3. Horas y días activos.

Se crea un diccionario con la información del dispositivo, y se llena con las variables obtenidas hasta el momento. Este diccionario será usado de aquí en adelante cuando se ocupe llamar algún dato.

```
device_info = {
    "name": device_name,
    "ip": device_ip,
    "active": is_active,
    "uptime_hours": uptime_hours,
}
```

Figura 4. Diccionario.

Se lee un valor del diccionario, y con un if se da un valor a otra variable, la cual es usada para mostrar si el dispositivo se encuentra activo o no (esto, para no realizar este proceso implícitamente a la hora de imprimir).

```
if device_info["active"] is True:
    status_msg = "ACTIVE"
elif device_info["active"] is False:
    status_msg = "DISCONNECTED"
else:
    status_msg = "UNKNOWN"
```

Figura 5. Estado actual del dispositivo.

A continuación se crea un documento de texto, y se escriben los datos del dispositivo en un reporte. Para cada dato, se lee la definición del diccionario, y se añade al documento. Al final, se añade el valor de la variable usada para guardar el estado actual del dispositivo.

```
filename = "device_report.txt"
with open(filename, "w") as report:
    report.write(
        "Network Device Report\n"
        f"Name : {device_info['name']}\n"
        f"IP : {device_info['ip']}\n"
        f"Ports : {device_info['ports']}\n"
        f"Uptime: {device_info['uptime_hours']} h ({device_info['uptime_days']} days)\n"
        f"Status: {status_msg}"
    )
```

Figura 6. Salida a archivo de texto.

También se escribe el resultado del reporte visiblemente en la consola, y se informa al usuario que el reporte ha sido generado y guardado con éxito.

```
print("Network Device Report:")
print(f"Name   : {device_info['name']}")
print(f"IP     : {device_info['ip']}")
print(f"Ports  : {device_info['ports']}")
print(f"Uptime: {device_info['uptime_hours']} h ({device_info['uptime_days']} days)")
print(f"Status: {status_msg}")
print("A file with the report has also been saved.")
```

Figura 7. Reporte finalizado.

Capítulo 3: Conclusión.

Después de haber realizado esta actividad, pudimos aprender a hacer uso de diversos elementos disponibles en Python, que nos servirán en el futuro a crear programas más complejos.